



universität
uulm

Interactive Computer Graphics

Chapter 7 – Visibility Determination

Summer Term 2026

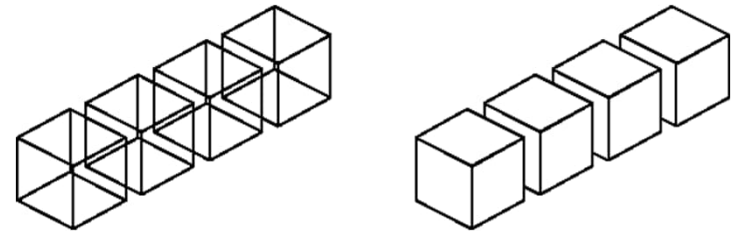
Timo Ropinski



Visual Computing Group
Institute of Media Informatics

Visibility Determination

- Problem
 - Given scene objects and virtual camera
 - Determine visible parts of scene objects
 - Not occluded by same scene object
 - Not occluded by other scene objects
 - Not pointing away from camera for closed, opaque objects
- Two equivalent perspectives on the problem
 - Determination of visible lines and surfaces (Visible Surface Determination, VSD)
 - Hidden Surface Removal (HSR) and Hidden Line Removal (HLR)
- Quality criteria for visibility determination algorithms
 - Generality: algorithms should be applicable for all types of geometry (e.g., convex/non-convex, polygons, polyhedra)
 - Performance: determining hidden scene object parts has to be more cost effective than their rasterization costs
 - Robustness: algorithms should be robust to numerical errors



Object-Based Algorithms

- Geometry-based analysis of scene objects before rasterization
 - Compare each scene object against all other scene objects
 - Determine visible parts based on virtual camera
 - Rasterize the visible scene parts

```
For each object in sceneObjects
    // Determine object parts that are visible from virtual camera's perspective
    visibleParts = determineVisibleParts(object, sceneObjects, virtualCamera)
    // Convert visible object parts into a format suitable for rendering
    rasterize(visibleParts)
EndFor
```

- Runtime complexity: $O(n^2)$, n number scene objects
 - Analytical visibility assessment
 - Example: Weiler Atherton algorithm

Image-Based Algorithms

- Pixel-based analysis of scene objects after rasterization
 - Rasterize scene objects based on image resolution
 - Investigate for each pixel which scene object is visible

```
For each pixel in outputRaster
    // Determine which scene object is visible at this pixel
    visibleObject = determineVisibleObject(pixel, sceneObjects)
    // Get the color of the visible object
    color = obtainObjectColor(visibleObject)
    // Set the pixel in the output raster to this color
    setPixel(pixel, color)
EndFor
```

- Runtime complexity: $O(n \cdot p)$, n number scene objects, p number pixels
 - Discrete visibility detection (depth sorting of pixel samples)
 - Example: z-buffer algorithm

Chapter Outline

7.1 Object-Based Algorithms

7.2 Binary Space Partitioning

7.3 Warnock Algorithm

7.4 Depth Buffer Algorithm

7.5 Depth Buffer Extensions

7.6 Ray Casting

References



universität
uulm

7.1 Object-Based Algorithms

Exploiting sorting and clipping



Visual Computing Group
Institute of Media Informatics

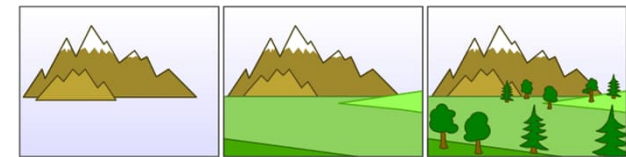
Painter's Algorithm 1/3



[Bob Ross – Grandeur of Summer](#)

Painter's Algorithm 2/3

- First algorithm for object-based visible polygon determination
 - Input: set of polygons
 - Output: rendering of visible polygons
- Procedure: simulate drawing process of a painting
 1. Sort polygons into list in back-to-front order
 2. Render sorted list from back to front



```
Procedure painter(Polygons P, Camera C)
    // Sort and draw polygons based on their depth relative to camera C
    SortedList Q
    For each polygon p in P
        depth z = calculateDepth(p, C)
        Insert p into Q with key z
    For each polygon q in Q
        drawPolygon(q) // Directly into the color buffer
EndProcedure
```

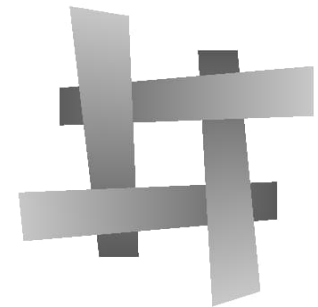
Painter's Algorithm 3/3

- Advantages

- Simple implementation
- No special graphics hardware necessary

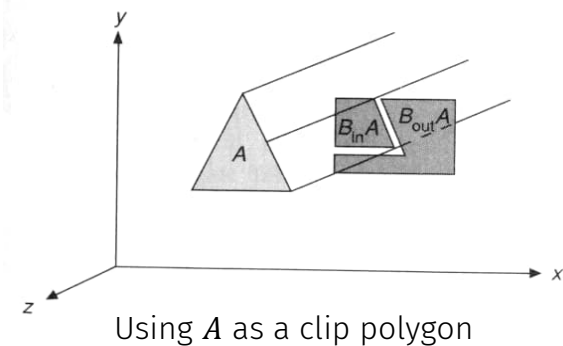
- Disadvantages

- Works for polygons only instead of 3D scene objects
- Application must implement sorting
- Depth values are in general not constant per polygon
 - Scene polygons must be subdivided to exclude mutual overlap



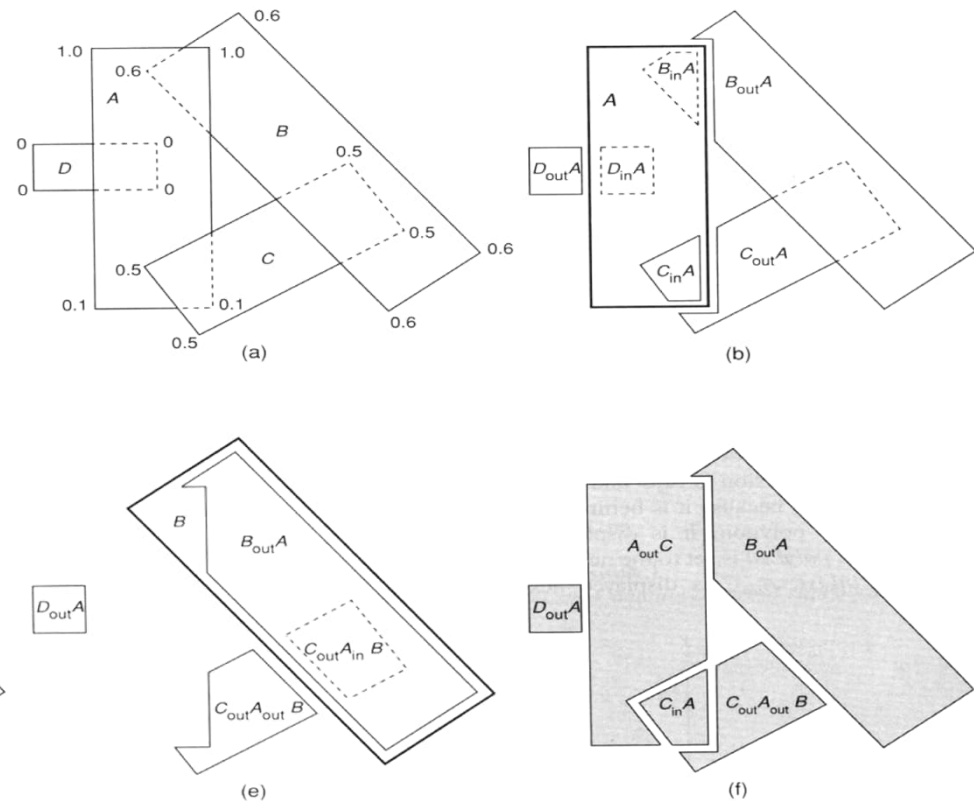
Weiler-Atherton Algorithm 1/3

- Clipping-based divide-and-conquer approach
 - Input: set of polygons
 - Output: set of visible polygon parts
- Procedure
 1. Sort polygons into list L in front-to-back order
 2. While polygon list L not empty
 1. Choose foremost polygon as clip polygon C
 2. Clip all other polygons against C via 2D clipping
 3. Move clipped polygons into *In* or *Out* list
 4. *Out* list will be new polygon list L



Weiler-Atherton Algorithm 2/3

- Example from original paper
 - Z value order flipped
 - Bold: clip polygon
 - Gray: unoccluded parts



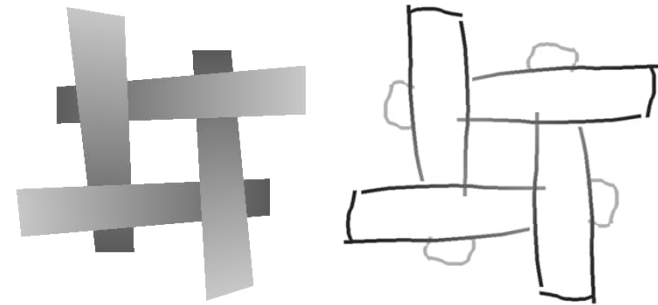
Weiler-Atherton Algorithm 3/3

■ Advantages

- Can handle lines and polygons
- Exact calculation of the visible polygons
- Can be used for wireframe rendering with different edge styles
- Rasterization and shading performed only for visible polygons

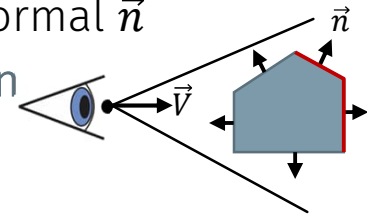
■ Disadvantages

- Badly parallelizable and thus badly supported by graphics hardware
- Implementation involved and vulnerable to numerical issues
- Output of smallest polygons can lead to image artifacts



Back Face Culling 1/2

- Solves visibility determination for one convex, opaque scene object, as its back faces are not visible
- Algorithm culls away polygons facing away from virtual camera
 - Input: convex, opaque 3D scene object
 - Output: set of front-facing polygons of input scene object
- Procedure: facilitate normal orientation to identify faces to be culled
 - Calculate dot product $s = \vec{v} \cdot \vec{n}$ of view direction $\vec{v}$ and face normal $\vec{n}$
 - If $s < 0$, face is pointing towards camera and should be **drawn** else if $s > 0$, face is pointing away and should be **culled**
- $\vec{n}$ can be calculated based on face vertices
 - $\vec{n}$ can be computed by taking cross product $\vec{n} = \vec{e}_0 \times \vec{e}_1$ of two face edges $\vec{e}_0, \vec{e}_1$
 - Normalize resultant vector to ensure unit length



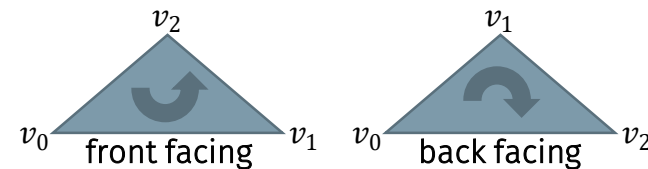
Back Face Culling 2/2

■ Advantages

- Simple implementation
- An average of 50% of the areas of a scene can be eliminated in this way
- Reduces number of polygons to be processed, thus reduces GPU transfers and speeds up the rendering process

■ Disadvantages

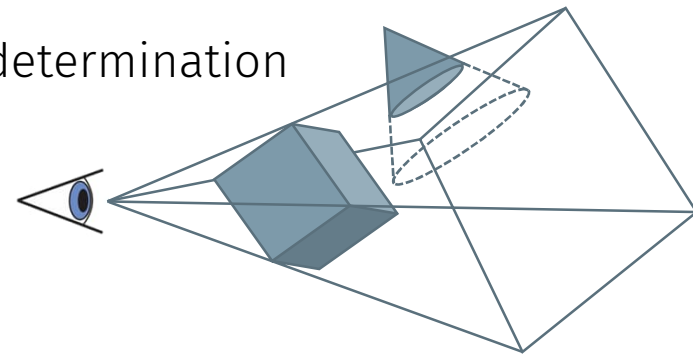
- Vertex order impacts face orientation
- Proper handling of non-convex or multiple scene objects
- Computation and transformation of normals can introduce overhead
- In dynamic scenes, where objects frequently move or rotate, maintaining correct normal orientations can be complex



Robert's Algorithm

- First algorithm for object-based visible line determination

- Input: set of convex 3D scene objects
- Output: set of visible lines

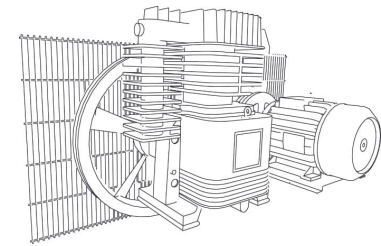


- Procedure: perform for all scene objects the following steps
 1. Remove self-occluded surfaces through back-face culling
 2. Compute shadow volume based on silhouette edges
 3. Clip edges against all computed shadow volumes
 4. Connect remaining edges per scene object

Assessment Robert's Algorithm

■ Advantages

- Determination of visible lines was particularly important for presentation on vector screens
- Today used in non-photorealistic rendering to generate line drawings



[cekuhnen, blenderartists.org]

■ Disadvantages

- Dissecting non-convex 3D scene objects into convex parts is difficult and requires high runtime complexity
 - Roger's extension: Use bounding box tests to reduce quadratic runtime complexity to linear



universität
uulm

7.2 Binary Space Partitioning

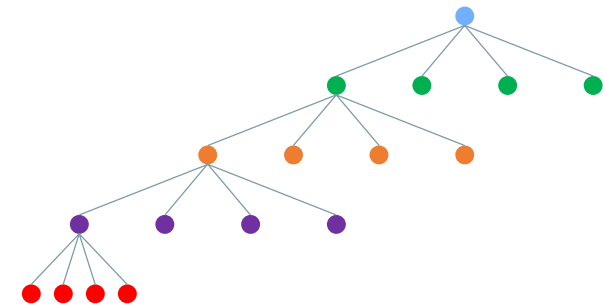
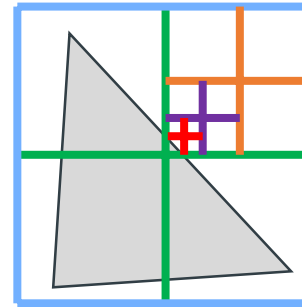
Hierarchical scene bisection



Visual Computing Group
Institute of Media Informatics

Spatial Data Structures

- Spatial data structures are specialized for organizing and storing scene data
- Types of spatial data structures
 - Quad Trees: Optimize 2D data retrieval by dividing space it into four nodes
 - Octrees: Optimize 3D data retrieval by dividing space it into eight nodes
 - *kd*Trees: Binary trees that organize points in *k*-dimensional space
 - BSP Trees: Binary trees that use planes to divide 3D space into convex subsets
- Importance in computer graphics
 - Efficiency: Enable real-time rendering through quick data access
 - Collision Detection: Enable determining interactions between scene objects
 - Visibility Determination: Helps deciding which scene parts are visible



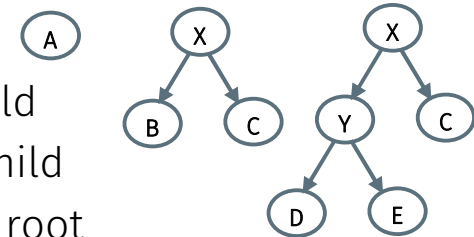
BSP Trees 1/2

- BSP Trees are Binary Space Partitioning trees that recursively subdivide space into convex subsets through hyperplanes
- Key functions
 - Space organization: divide 3D space into subspace represented as nodes
 - Query efficiency: facilitate rapid querying like determining object visibility
 - Render optimization: Render scenes efficiently through occlusion culling
- Key properties
 - Recursive, hierarchical subdivision scheme
 - Each node can store one or more polytopes

BSP Trees 2/2

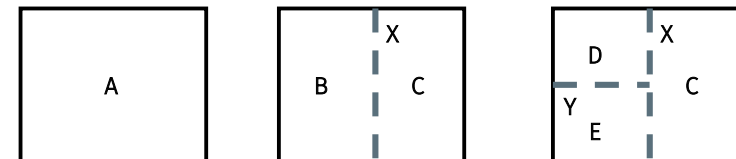
■ Definition binary tree

- A binary tree is a tree with at most two child nodes per node
- Different traversal schemes for binary trees exist
 - In-order traversal: first left child, then root, then right child
 - Pre-order traversal: first root, then left child, then right child
 - Post-order traversal: first left child, then right child, then root



■ Definition hyperplane

- A hyperplane in the n -dimensional space is a $(n-1)$ -dimensional object that separates the n -dimensional space into two halves
 - In 2D space, a line is a hyperplane
 - In 3D space, a plane is a hyperplane



Node Data Structure

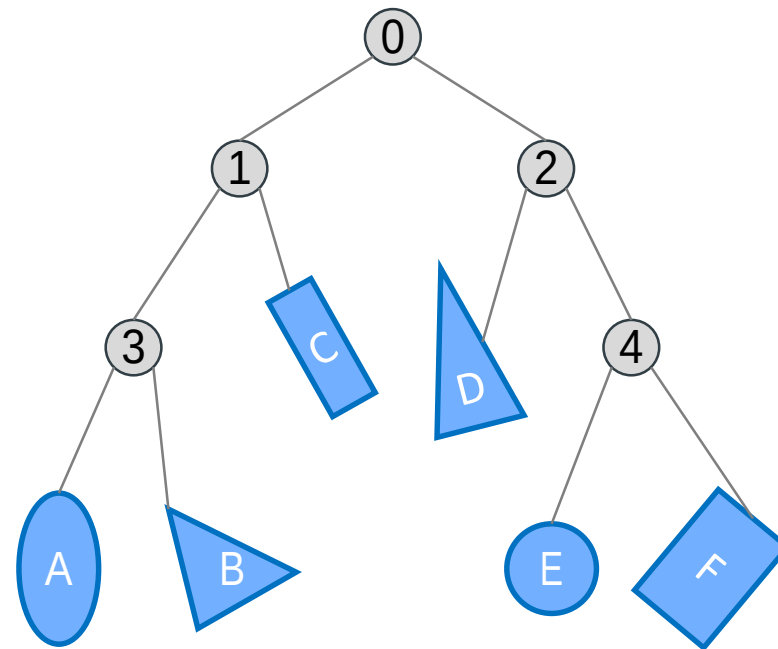
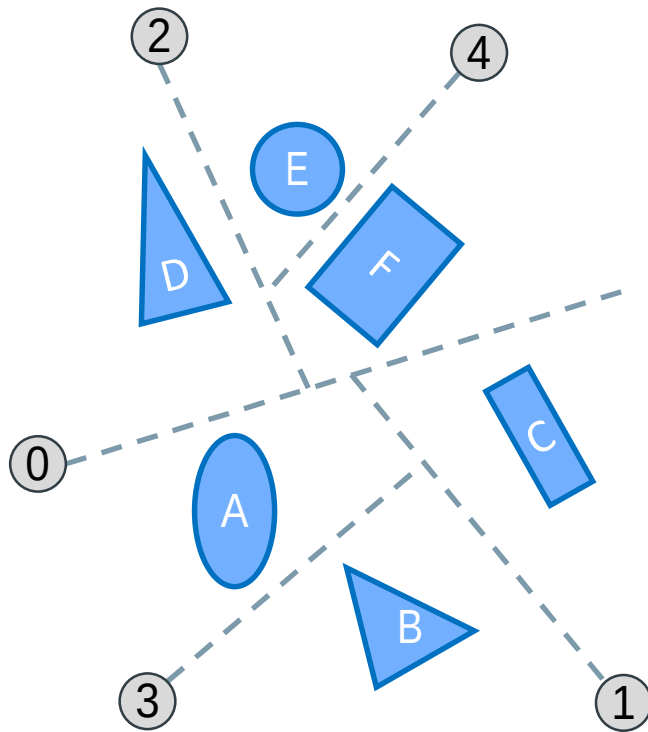
- Each node contains
 - Hyperplane
 - List of scene geometry associated with the node
 - References to the two subspaces

```
Class BSPNode:  
    Declare plane as Plane  
    Declare geometry as List of Polygon  
    Declare front as BSPNode  
    Declare back as BSPNode
```

BSP Tree Construction

- Choose a hyperplane from scene geometry for root node and associate that plane with root node, split remaining geometry at plane
- Proceed recursively with the two resulting sets of scene geometries and check each polygon against hyperplane
 - Coinciding: associate with node
 - Intersecting: subdivide into two parts and sort into half-spaces
- Termination criterion for recursion
 - Stop if a node has fewer than a specified number of polygons
 - Stop when every single polygon is assigned to a node

BSP Tree Construction - Example



BSP Tree Construction – Pseudo Code

```
Function buildBSPTree(BSPNode node, List of Polygons list)
// Select one polygon to use as a hyperplane
Polygon p = Get first polygon from list
Set node.plane to the plane of polygon p
Add polygon p to node.geometry
// Initialize lists for front and back polygons
Declare frontList, backList as List of Polygons
// Distribute remaining polygons into the corresponding half-spaces
While there is another polygon p in list
    Evaluate the spatial relation of polygon p with respect to node.plane
    Switch (relation)
        Case COINCIDENT:
            Add p to node.geometry
        Case INFRONT:
            Add p to frontList
        Case BACK:
            Add p to backList
        Case SPANNING:
            Split polygon p by node.plane, distribute pieces to frontList and backList
// Recursively build the BSP tree in both half-spaces
Set both node.front and node.back to null
If frontList is not empty
    Create a new BSPNode for node.front
    Call buildBSPTree recursively for node.front with frontList
If backList is not empty
    Create a new BSPNode for node.back
    Call buildBSPTree recursively for node.back with backList
```

Visibility Determination

- Visibility determination through modified depth-first traversal respecting the current camera
 - Tree traversal handles the entire scene geometry
 - Drawing is done in analogy to painter's algorithm
 - Traversing provides the necessary "strict back-to-front order"

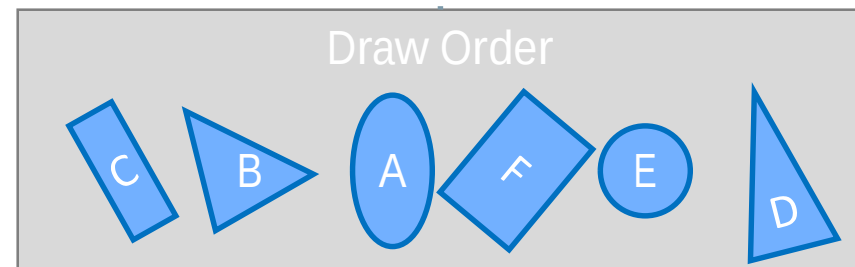
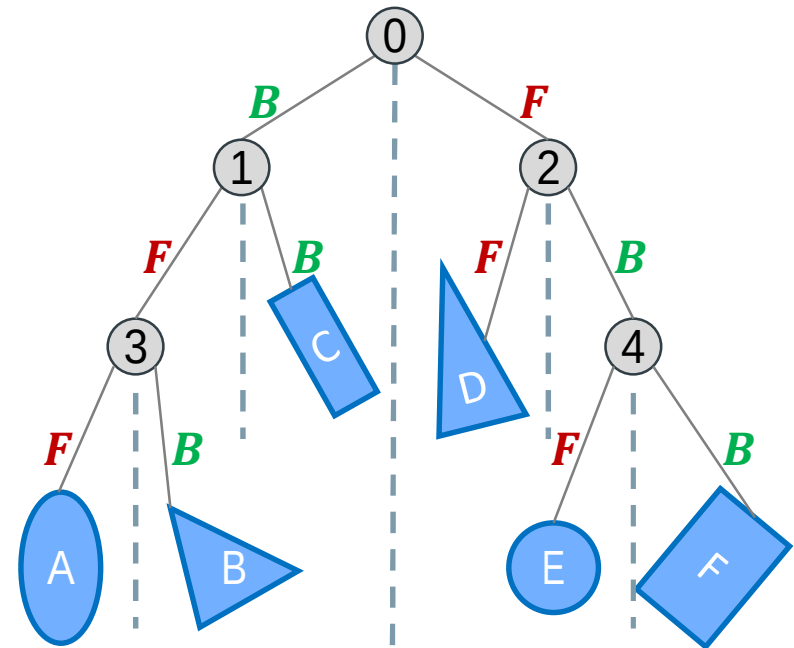
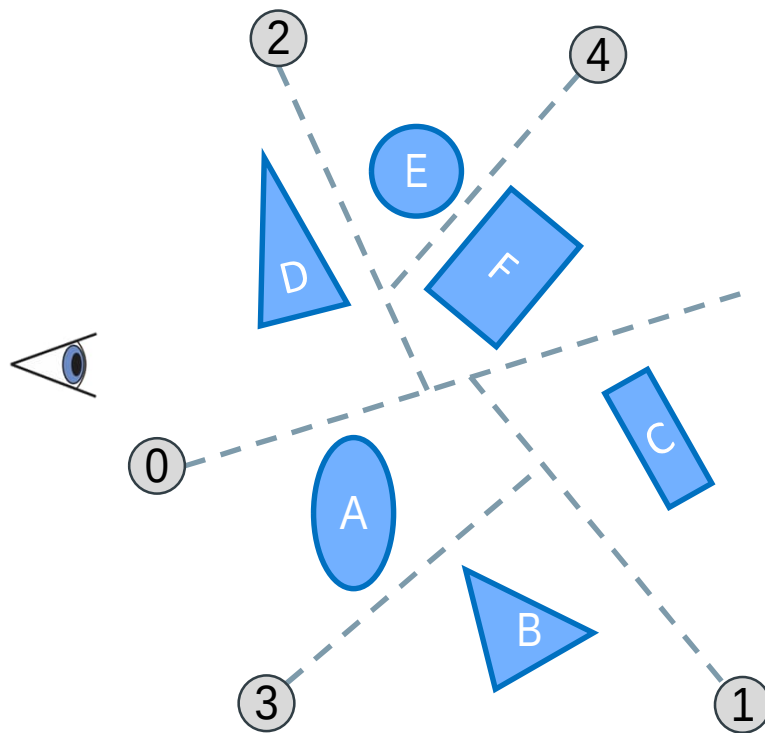
- For every node
 - Draw the back part of the scene with respect to the camera location first
 - Draw the polygon in the current layer
 - Draw the front part of the room relative to the camera location

Visibility Determination – Pseudo Code

```
Function BSPHiddenSurfaceTraversal(BSPNode node, Point eye)
    // Calculate the signed distance from the eye point to the partition plane
    float distance = classifyPoint(node.plane, eye)

    If distance < 0 Then
        // Eye-point is on the back side of the plane
        Call BSPHiddenSurfaceTraversal for node.front with eye
        Draw the geometry in node
        Call BSPHiddenSurfaceTraversal for node.back with eye
    Else If distance > 0 Then
        // Eye-point is in front of the plane
        Call BSPHiddenSurfaceTraversal for node.back with eye
        Draw the geometry in node
        Call BSPHiddenSurfaceTraversal for node.front with eye
    Else
        // Eye-point is exactly on the partition plane
        Call BSPHiddenSurfaceTraversal for node.back with eye
        Call BSPHiddenSurfaceTraversal for node.front with eye
```

BSP Tree Traversal - Example

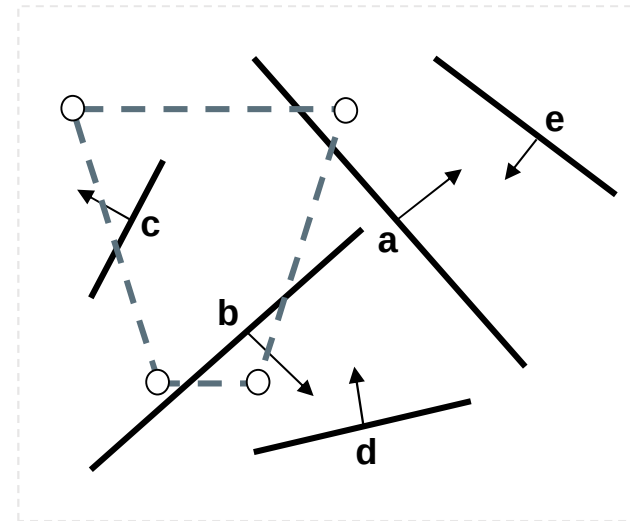


Splitting Planes Selection

- Hyperplane selection influences tree balance and required geometry splits
- Two alternative approaches exist
 - Planes can be derived from scene geometry
 - Planes can be chosen freely
- Useful heuristics
 - Choose planes that are also polygon planes
 - Choose large polygons first
 - Choose planes that don't split many polygons
 - Try to choose planes that evenly divide the data (generate few cells)

Traversal Optimization

- Sub-trees can be skipped during traversal if content is outside view frustum
 - Testing is done by checking 8 corner points of 3D view frustum
 - If there are two points on different sides, then the subtree is traversed
 - If all points are in one subspace, then the other subspace does not have to be traversed (view-frustum culling)



Assessment BSP Trees

■ Advantages

- Simple implementation
- Do not need a depth buffer
- Applicable to static and dynamic scenes
(static scene geometry precalculated, dynamic objects inserted)
- Support painter's algorithm by ensuring ordering

■ Disadvantages

- Tree balance difficult (i.e., choice of splitting planes)
- BSP tree construction must be redone for dynamic scenes
- Tree traversal cannot be parallelized
⇒ BSP trees poorly supported by graphics hardware



universität
uulm

7.3 Warnock Algorithm

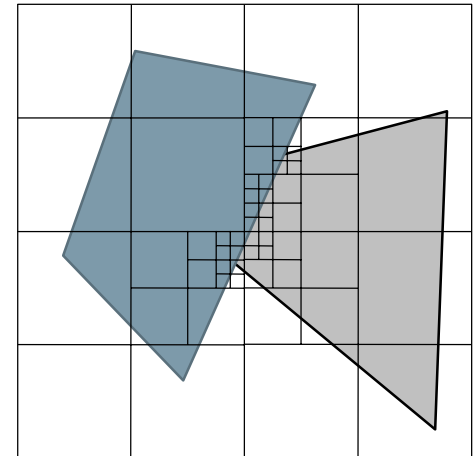
Image-based divide-and-conquer strategy



Visual Computing Group
Institute of Media Informatics

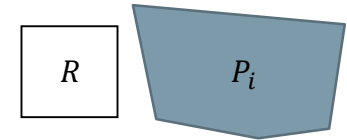
Warnock Algorithm

- Hybrid object- and image-based divide-and-conquer approach
 - Input: set of polygons and viewport
 - Output: set of visible polygon parts
- Procedure: exploit spatial coherence of projected polygons to divide viewport
 - Divide viewport recursively into squared regions until visibility determination for a region is trivial
 - Recursion depth increases in viewport parts where polygons intersect

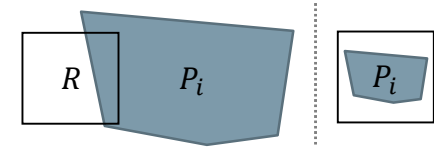


Region-Based Visibility Determination

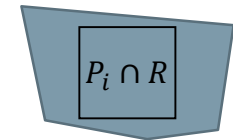
- Given viewport region R and set of polygons P , for the following cases visibility determination can be considered trivial



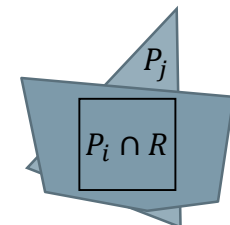
Case 1



Case 2



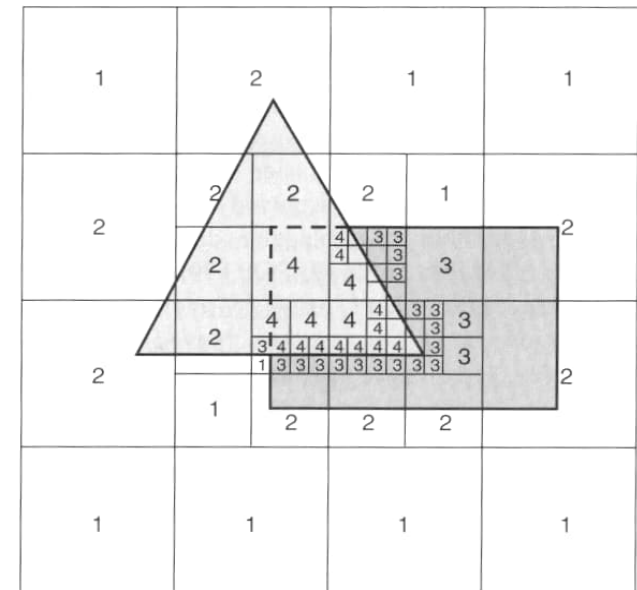
Case 3



Case 4

Procedure

- Start with entire viewport region R
- Subdivide R into four equal subregions
 - Terminate if subregion size matches pixel size
 - Classify subregions based on trivial cases
 - If subregion matches trivial cases: resolve otherwise: recursively subdivide subregion



Example run with case numbers

Pseudo Code

```
Procedure warnockHSR(Polygons list, Viewport vp)
  If isSimple(list, vp) Then
    // Base case: Simple enough scenario to draw directly
    drawPolygons(list)
  Else
    // Recursive case: The viewport is too complex and needs to be subdivided
    Declare vp1, vp2, vp3, vp4 as Viewport
    // Subdivide the current viewport into four equal parts
    subdivideViewPort(vp, vp1, vp2, vp3, vp4)

    // Build lists of polygons that are relevant for each subdivided viewport
    List<Polygon> list1 = selectPolygonsInViewport(list, vp1)
    List<Polygon> list2 = selectPolygonsInViewport(list, vp2)
    List<Polygon> list3 = selectPolygonsInViewport(list, vp3)
    List<Polygon> list4 = selectPolygonsInViewport(list, vp4)

    // Recursively apply the Warnock algorithm to each subdivided viewport
    warnockHSR(list1, vp1)
    warnockHSR(list2, vp2)
    warnockHSR(list3, vp3)
    warnockHSR(list4, vp4)
  End If
End Procedure
```

Assessment Warnock Algorithm

- Advantages

- Efficient for large polygons by exploiting spatial coherence
- Easy to implement through recursive programming

- Disadvantages

- Restriction to polygons as primitives
- Investigation of non-trivial cases numerically difficult
- No graphics hardware support



universität
uulm

7.4 Depth Buffer Algorithm

Exploiting image storage for visibility determination



Visual Computing Group
Institute of Media Informatics

Depth Buffer Algorithm 1/3

- Image-based visibility determination algorithm that exploits extra graphics memory called depth buffer (= z-buffer)

- Depth buffer
 - Depth buffer is a 2D raster that contains depth values
 - Depth value calculation happens during rasterization
 - Depth values are normalized distance of a visible fragment to the near plane in normalized device coordinates
 - Depth buffer is part of the frame buffer, (i.e., for each pixel, depth value is stored in addition to color values)
 - Depth buffers have in general between 16 and 32 bit accuracy

Depth Buffer Algorithm 2/3

- Depth buffer algorithm follows occlusion culling principle

```
Function occlusionCulling(List of Polygons scene)
```

```
    Initialize an OcclusionRepresentation OR
```

```
    For each object obj in scene
```

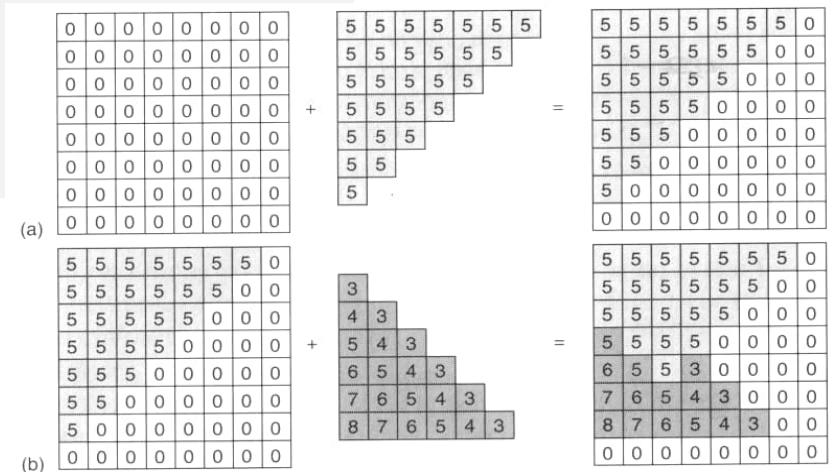
```
        If obj is occluded by OR Then
```

```
            Continue to the next iteration (skip this object)
```

```
        Else
```

```
            Render obj
```

```
            Update OR with obj
```



Depth Buffer Algorithm 3/3

- Depth buffer algorithm performs depth testing for each fragment
 - Depth buffer is initialized with background value (e.g., $z = 1$)
 - For every scene object
 - Rasterize scene object
 - For each fragment
 - Calculate z -value z
 - If z is smaller than current depth value at fragment location in depth buffer
 - Write z into depth buffer
 - Write fragment color into color buffer
 - Otherwise
 - Ignore fragment and leave framebuffer values (z and color) unchanged (i.e., fragment lies behind an earlier, already drawn fragment)

Pseudo Code – Depth Buffer Test

```
Function zBufferHSR(List of Geometry list)
    // Initialize z-buffer and color buffer for each pixel on the screen
    For x from 0 to Width
        For y from 0 to Height
            Set depthbuffer[x][y] to 1.0 // Maximum depth value
            Set colorbuffer[x][y] to backgroundColor // Default background color

    // Evaluate all objects in the scene
    For each object obj in list
        // Rasterize the object into fragments
        List of Fragments fragments = rasterize(obj)

        // Test each fragment against the z-buffer
        For each fragment f in fragments
            If f.z < depthbuffer[f.x][f.y] Then
                Set depthbuffer[f.x][f.y] to f.z // Update depth buffer with closer fragment
                Set colorbuffer[f.x][f.y] to f.color // Update color buffer with fragment's color
```

OpenGL Depth Buffer

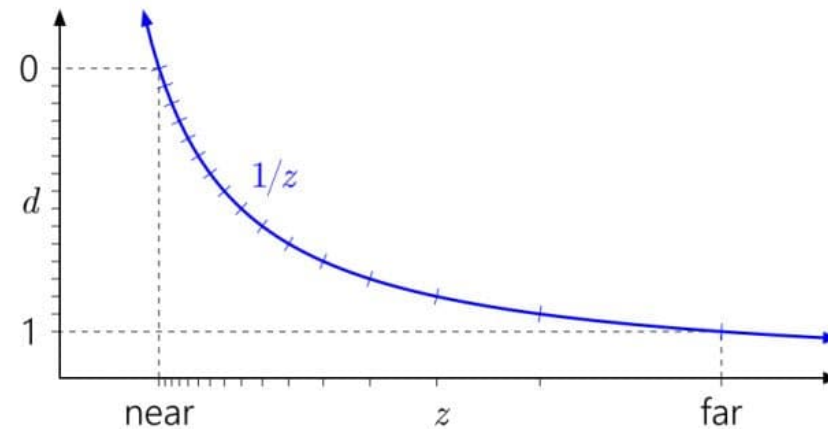
- OpenGL depth buffer: initialization and activation
 - Depth values are limited to $[0, 1]$ (0: near-plane, 1: far-plane)
 - Depth test must be explicitly activated
 - Depth test comparison function can be changed

```
Function init()
    glClearDepth(1.0); // Initializes maximum z-buffer depth
    glEnable(GL_DEPTH_TEST); // Enables depth testing
    glDepthFunc(GL_LESS); // Fragments pass if they are closer
    glClearColor(bgR, bgG, bgB, bgA); // Sets background color
    // Other initialization settings as needed
    ...

Function draw(List of Geometry scene)
    glClear(GL_DEPTH_BUFFER_BIT); // Clear the depth buffer
    Draw scene geometry
    // Additional rendering operations as needed
    ...
```

Depth Buffer Precision

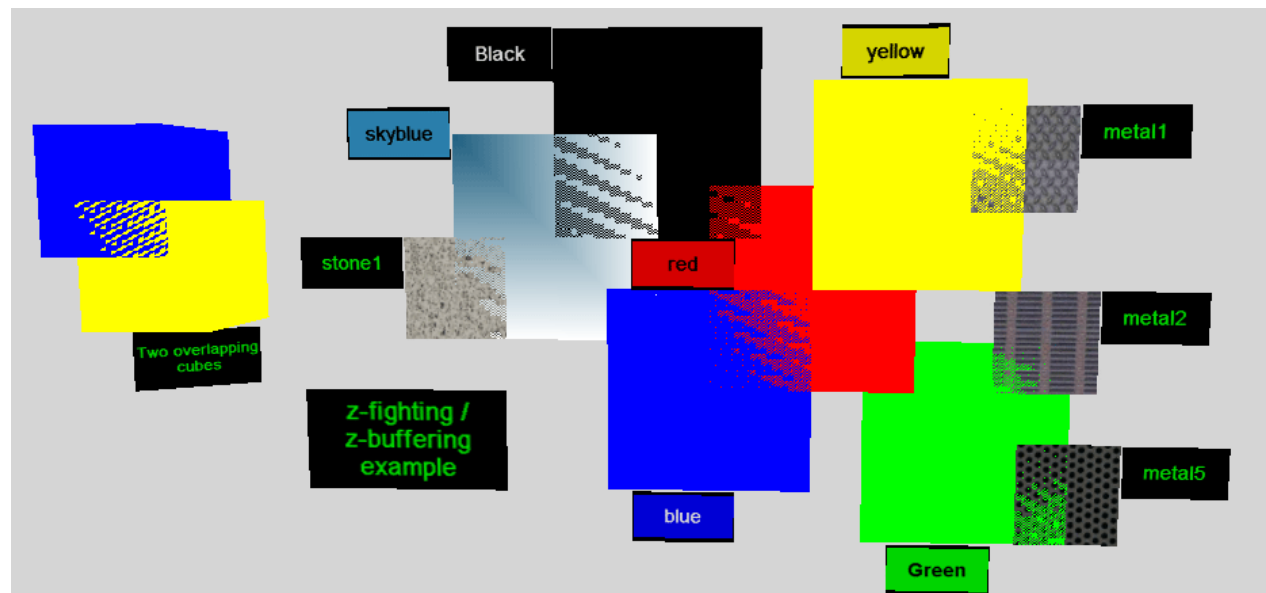
- Depth buffer precision is limited by its bit-depth (e.g., 16-bit, 24-bit, 32-bit), which determines how finely depth can be distinguished
- Depth buffer precision is not linear: denser at near- and sparser at far-clipping plane $\Rightarrow$ depth conflicts can occur especially at greater distances



[Nvidia Blog](#)

Z-Fighting 1/2

- Z-fighting occurs when two or more objects are very close together in depth
 - Depth buffer can't consistently resolve which one is closer causing flickering or overlapping artifacts in the rendered scene



Z-Fighting 2/2

- Mitigation through depth buffer properties
 - Use higher precision buffers by switching from 24-bit to 32-bit depth
 - Adjust near and far planes: bringing near plane out- and far plane inwards reduces the depth range, improving effective depth buffer precision
- Mitigating through rendering properties
 - Polygon offset: slightly offset surfaces that might otherwise z-fight, commonly used for rendering shadows or decals
 - Staggered rendering: render scenes in multiple passes with adjusted depth ranges to ensure that precision is focused where it is most needed

Assessment Depth Buffering

■ Advantages

- Easy implementation
- No presorting of scene objects required
⇒ Object evaluation can be performed in any order
- Not limited to polygonal geometries
⇒ Suitable for all rasterizable objects (also curves, fractals, etc.)
- Hardware support even with low-cost hardware
- Separate storage of depth values enables storage of depth images

■ Disadvantages

- Depth buffer necessary as additional resource
- Semi-transparent scene objects not supported
- Unnecessary rasterization of non-visible scene geometry
- Limited numerical precision of the depth buffer
- Nonlinear distribution of depth accuracy

7.5 Depth Buffer Extensions

How to render semi-transparent scene objects, and how to generate halos



Visual Computing Group
Institute of Media Informatics

Semi-Transparent Objects 1/2

- Rendering of semi-transparent objects requires special handling
 - Opaque and semi-transparent geometry need to be separated
 - Opaque geometry is rendered first, semi-transparent blended on top

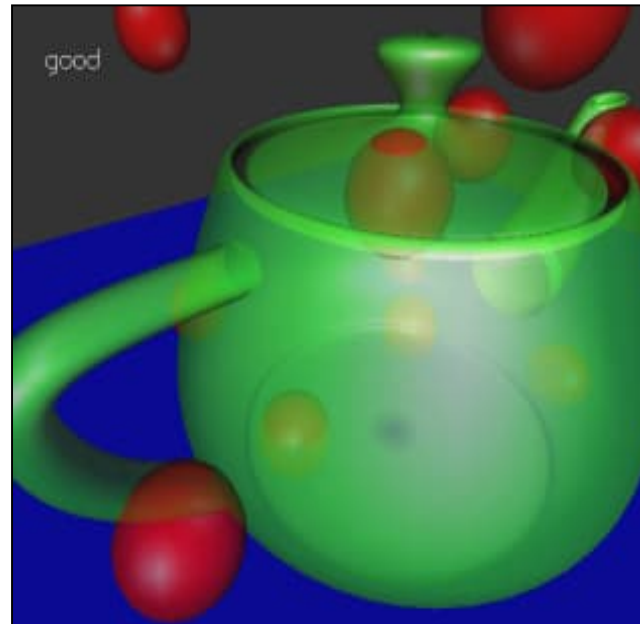
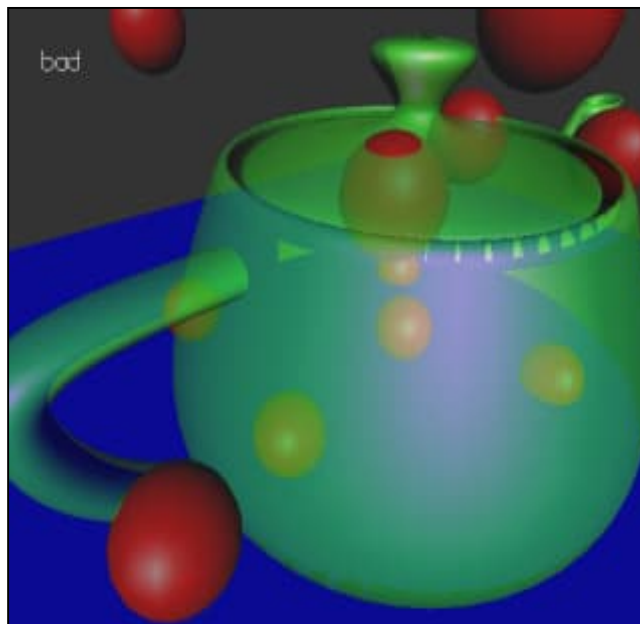
```
Function draw(Vector scene)
    // Clear the framebuffer to prepare for new drawing
    Clear buffers (depth and color)

    // First rendering pass: Draw all opaque objects
    DrawOpaqueGeometry(scene)

    // Second rendering pass: Draw transparent objects
    Enable color blending
    Set blending function to GL_SRC_ALPHA and GL_ONE_MINUS_SRC_ALPHA
    Lock depth buffer to prevent updates during transparent object rendering
    DrawTransparentGeometry(scene) // Render transparent objects
    Unlock depth buffer to allow depth updates
    Disable color blending
```

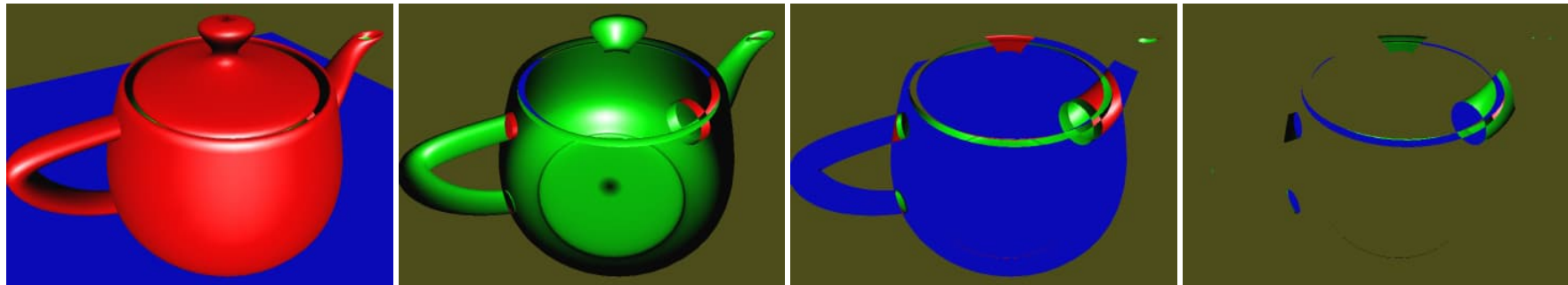
Semi-Transparent Objects 2/2

- Issue: correct transparency without presorting of scene objects not possible



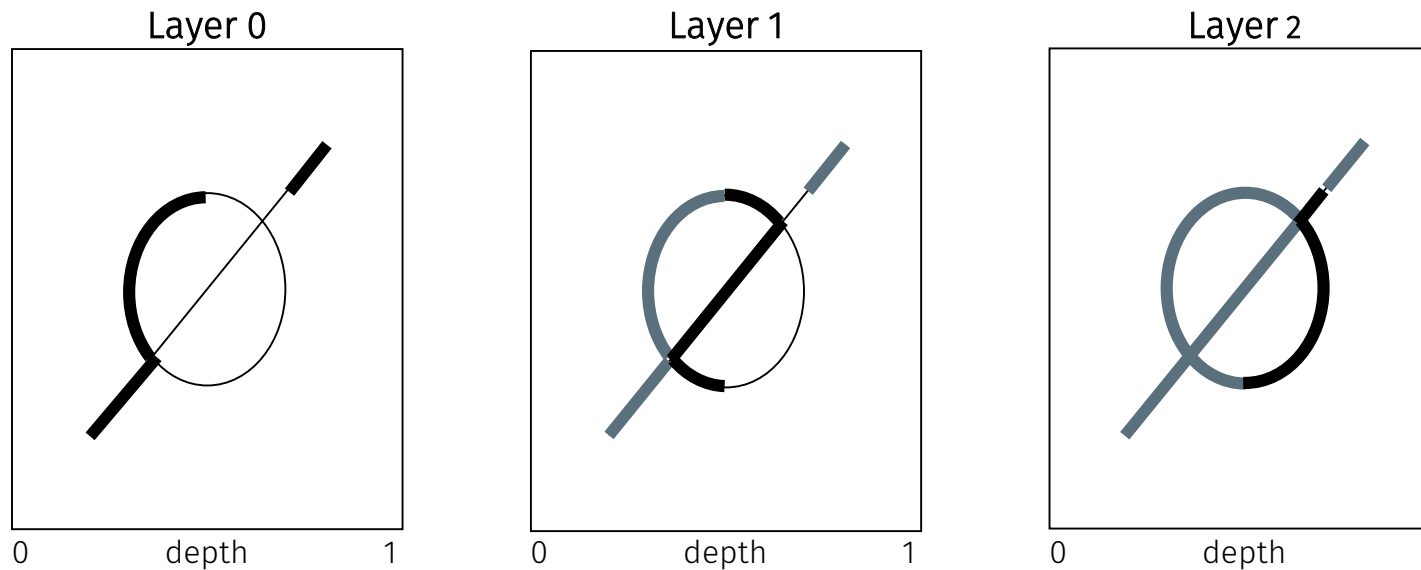
Order-Independent Transparency 1/4

- Order-independent transparency exploits depth peeling to avoid sorting
- Depth peeling peels away depth layers one-by-one in front-to-back order
 - Extract nearest fragments with smallest z values in first rendering pass
 - Extract next further fragments in each subsequent rendering pass
 - The n th rendering pass delivers the fragments to the n th depth plane



Order-Independent Transparency 2/4

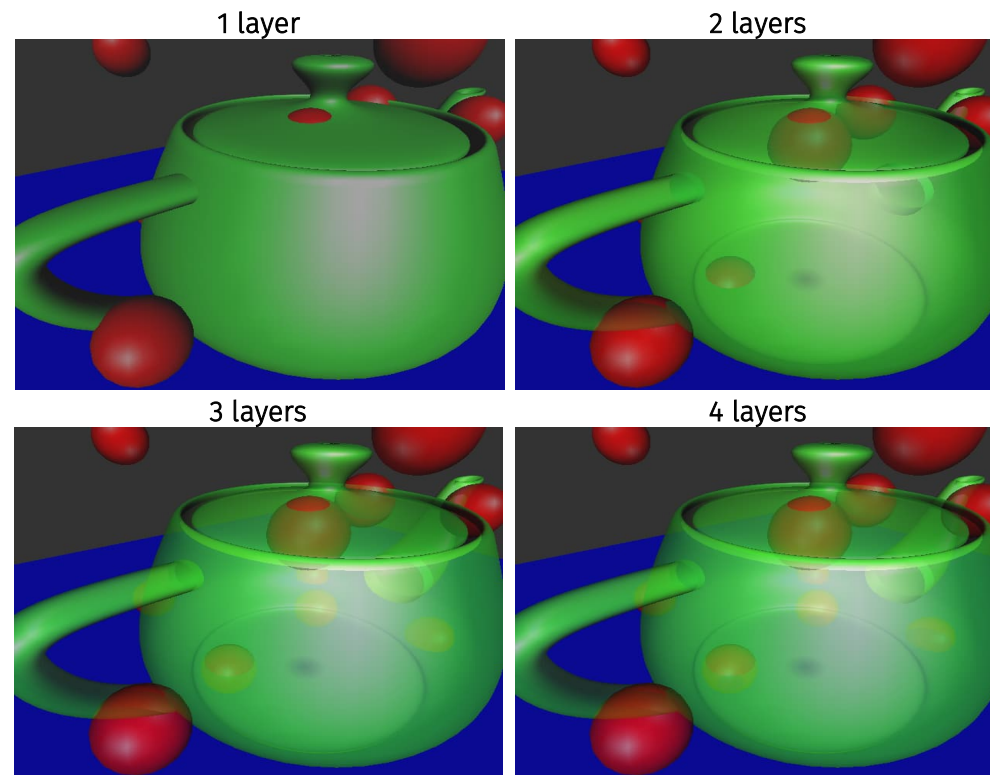
- Illustration of subsequent layers



Order-Independent Transparency 3/4

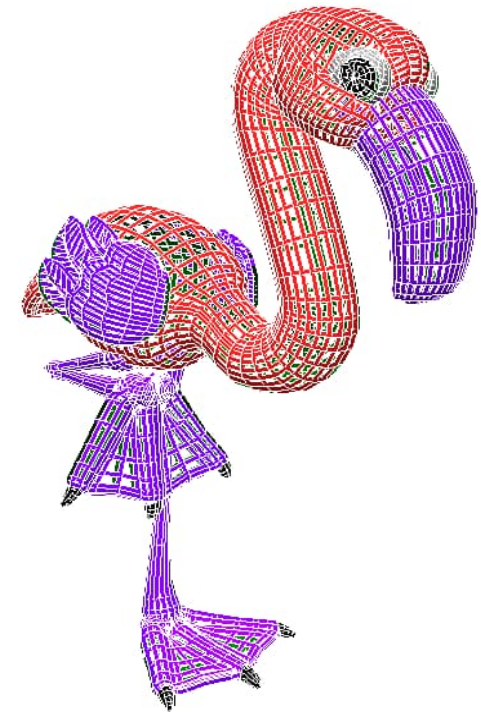
- 1st pass
 - Render scene with conventional depth test (`GL_LESS`)
 - Store resulting depth layer in Z-Buffer A
 - Store resulting color layer as Layer 0
- 2nd pass
 - Render scene with conventional depth test (`GL_LESS`), and use second depth test on read-only Z-Buffer A (`GL_GREATER`)
 - Store resulting depth layer in Z-Buffer B
 - Store resulting color layer as Layer 1
- 3rd pass
 - Render scene with conventional depth test (`GL_LESS`), and use second depth test on read-only Z-Buffer B (`GL_GREATER`)
 - Store resulting depth layer in Z-Buffer A
 - Store resulting color layer as Layer 2
- ...
- Postprocess: combine Layer 0 to N in color buffer (screen-aligned textures)

Order-Independent Transparency 4/4



Generating Haloes 1/2

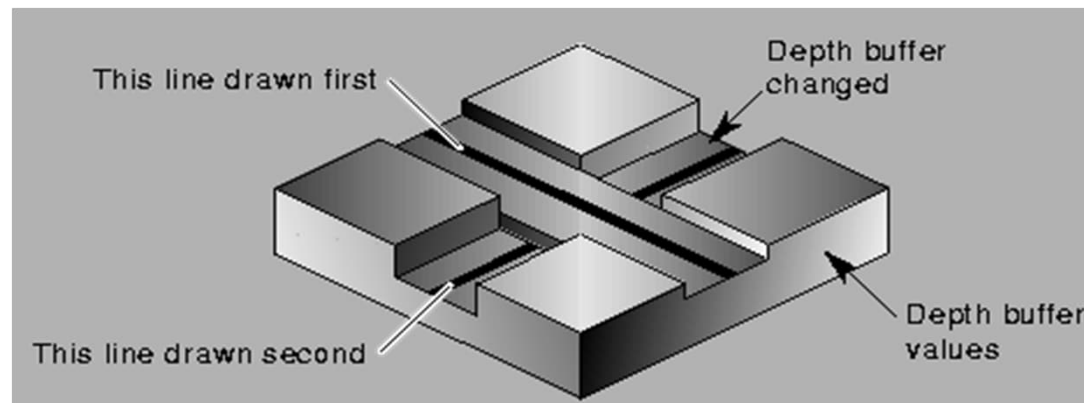
- Halo: circle of light around sun or moon caused by ice crystals in the air; [...]; a distinguishing zone surrounding a central object
- Haloes as wireframe illustration tools result in better spatial comprehension
 - Add highlights to foreground lines
 - Add darkening to background lines



Generating Haloes 2/2

■ Proceeding

- Render wireframe model with **thick lines** into depth buffer without writing to the color buffer $\Rightarrow$ depth image contains thick lines
- Render wireframe model with **thin lines** into color buffer $\Rightarrow$ Lines of foreground surfaces have a "safety margin"





universität
uulm

7.6 Ray Casting

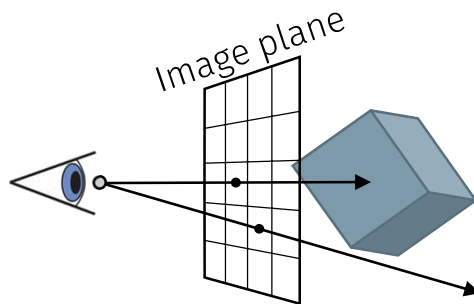
Shooting rays to determine visibility



Visual Computing Group
Institute of Media Informatics

Ray Casting Principle

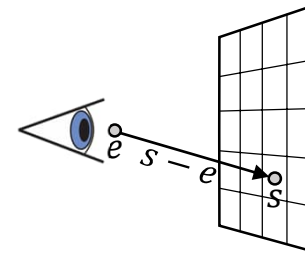
- Image-based visibility determination algorithm exploiting ray intersections
 - Ray initialization - initialize ray from viewpoint through each pixel (ray generation)
 - Intersection calculation - find object intersection closest to camera (ray intersection)
 - Shading - determine pixel color based on intersection color (shading)



```
// For-loop over all pixels in the image
For each pixel in the image
  Initialize ray from the viewpoint through the pixel
  Find the first intersection point of the ray with the scene
  Compute the shading for the pixel based on the intersection point
  Set the color of the pixel based on the computed shading
```

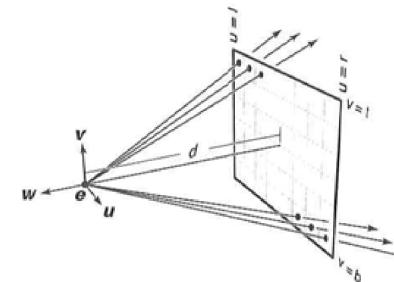
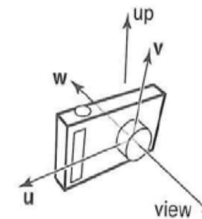
Ray Initialization 1/3

- Ray is defined by origin and direction
- Mathematically parameterized as a 3D line via t
 - $p(t) = e + t \frac{s-e}{|s-e|}$
 - e : ray origin
 - $s - e$: ray direction
- Ray properties
 - $p(0) = e$ and $p(1) = s$
 - $0 < t_1 < t_2 \Rightarrow p(t_1)$ closer to e than $p(t_2)$
 - $t < 0 \Rightarrow p(t)$ behind origin



Ray Initialization 2/3

- How can we find s ?
- Assumption: we have a simplified camera model
 - Positioned in e
 - Orthonormal base $\{u, v, w\}$
 - Orthonormal: orthogonal + normalized
 - $\{u, v, w\}$ defines right-handed coordinate system
- Specification of the image plane in relation to $\{u, v, w\}$
 - l and r define left and right borders relative to e towards u (typical: $l < 0 < r$)
 - b and t define lower and upper borders relative to e towards v (typical: $b < 0 < t$)
 - Distance between image plane and e is d



Ray Initialization 3/3

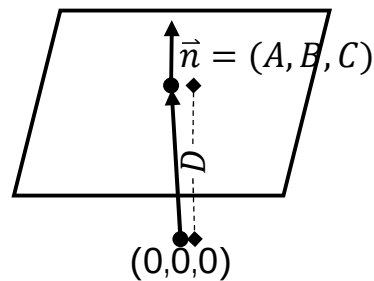
- Definition of the pixel grid with $n_x \times n_y$ pixels
 - Size of the pixel raster is $(r - l) \times (t - b)$
 - Pixel spacing is
 - x-coordinate: $(r - l)/n_x$
 - y-coordinate: $(t - b)/n_y$
 - To center grid in $(r - l) \times (t - b)$ a $\frac{1}{2}$ pixel wide border is assumed
- Calculate pixel position (x, y) in camera coordinates
- $s_u = l + (r - l) \cdot \left(x + \frac{1}{2}\right) / n_x$
- $s_v = b + (t - b) \cdot \left(y + \frac{1}{2}\right) / n_y$
- $s_w = -d$
- Calculate ray direction
 - $s - e = s_u \cdot u + s_v \cdot v + s_w \cdot w$

Intersection Calculation

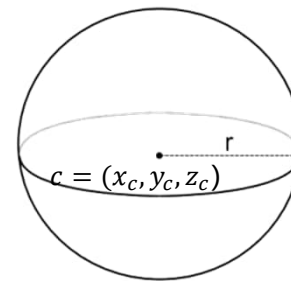
- Find smallest positive t for which $p(t)$ is an intersection
- Simple procedure
 - Iterate over all scene objects and calculate intersections
 - Sort intersections
 - Select intersection with smallest t
- Intersection calculation is executed $n_x \cdot n_y \cdot n_o$ times
- Optimized intersection calculation necessary

Implicit Surface Intersection Calculation

- Find set of all points with $f(x, y, z) = 0$
- Examples: plane, sphere, quadrics, ...



$$f(x, y, z) = Ax + By + Cz + D$$



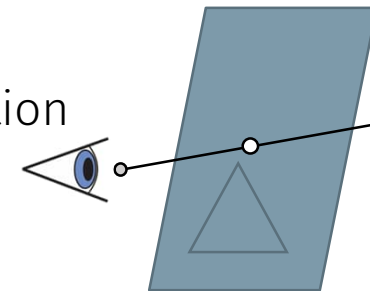
$$f(x, y, z) = (x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 - r^2$$

- Intersection calculation: find $f(e + t(s - e)) = 0$

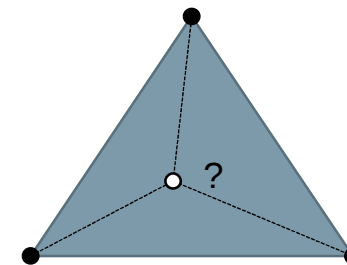
Intersection Calculation - Triangle

- Idea: find intersection with plane which embeds triangle, and test whether intersection lies inside triangle

1. Calculate intersection with plane through implicit surface intersection calculation



2. Exploit barycentric coordinates to determine if intersection lies inside triangle



Pseudo Code – Analytic Intersection Computation

```
Function castRay(List sceneObjects, Window win, Camera cam)
    // Iterate over each pixel on the window
    For x from 0 to win.width
        For y from 0 to win.height
            Initialize ray r = calculateRay(x, y, win, cam) // Calculate ray from camera through pixel (x, y)

            Initialize closestIntersection as null
            Initialize closestObject as null

            // Test intersection of ray with each object in the scene
            For each object obj in sceneObjects
                intersection = calculateIntersection(obj, ray)
                If intersection occurs and (closestIntersection is null or intersection is closer than closestIntersection)
                    Update closestIntersection with this intersection
                    Set closestObject to obj

            // If an intersection was found, determine the pixel's color based on the closest object
            If an intersection was found
                color = determineColor(closestObject, intersection)
                Set color of pixel (x, y) to this color
```

Pseudo Code - Iterative Intersection Calculation (Ray Marching)

- Ray march in ϵ steps and check all scene objects for intersection

```
Function intersect(const Ray& ray, const List of SceneObjects& sceneObjects) const
    Initialize t as 0.0 // Starting point for ray parameter
    Initialize epsilon as 0.01 // Small increment for ray parameter

    // Loop to incrementally trace the ray through the scene
    Loop for a large number of iterations (up to BIG_NUMBER)
        Calculate rayPos as ray.origin + ray.direction * t // Current position along the ray
        // Check for intersections with all scene objects
        For each sceneObject in sceneObjects
            If sceneObject intersects with rayPos
                Set result.position to ray.origin + ray.direction * t // Intersection position
                Set result.color to sceneObject.color // Color at the intersection
                Set result.normal to sceneObject.normal // Normal at the intersection
                Return result // Return the intersection details
        Increment t by epsilon // Move further along the ray
    Return result // Return the result with default values if no intersection is found
```

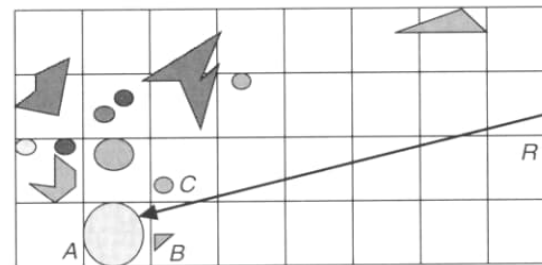
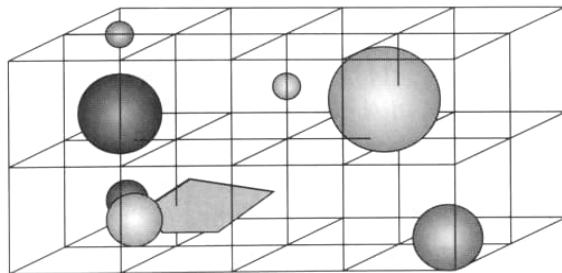
Assessment Ray Casting

■ Advantages

- Image-precise algorithm that support all types of scene objects
- Analytic intersection calculation possible
- Can be parallelized and supported by graphics hardware

■ Disadvantages

- Intersection calculations are computed often
 - But: spatial data structures can be used to reduce intersection calculations



Chapter Summary

- Visibility determination ensures only scene objects visible from camera are rendered
- Effective visibility determination is crucial for rendering efficiency and visual accuracy
- Object-based algorithms perform geometry-based analysis prior to rasterization
 - Examples include the Painter's Algorithm, Weiler-Atherton, and Back Face Culling
 - These methods often suffer from higher complexity and can be poorly parallelizable but are crucial for accurate visibility determination
- Image-based algorithms operate after scene objects are rasterized based on pixel-level
 - The z-buffer algorithm is a fundamental image-based method that exploits depth values
 - Simpler and often hardware-accelerated, but methods often constraint by pixel raster



universität
uulm

References

Literature and other sources used in this chapter



Visual Computing Group
Institute of Media Informatics

References

■ Text Books

- Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and Practice, Addison-Wesley.

■ Papers

- Everitt, C. (2001). Interactive order-independent transparency. White paper, nVIDIA, 2(6), 7.
- Roberts, L. G. (1963). Machine perception of three-dimensional solids (Doctoral dissertation, Massachusetts Institute of Technology).
- Sutherland, I. E., Sproull, R. F., & Schumacker, R. A. (1974). A characterization of ten hidden-surface algorithms. ACM Computing Surveys (CSUR), 6(1), 1-55.
- Warnock, J. E. (1969). A hidden surface algorithm for computer generated halftone pictures. The University of Utah.